A decorative graphic on the left side of the slide, consisting of a black crosshair overlaid on a blue, red, and yellow square.

Computer Programming II (CSE2035)

Week 6 Lab Session:

Object Oriented Programming Part 2

Prof. Jaeseung Choi
Dept. of Computer Science and Engineering
Sogang University

Homework 6-1: Lecture Class

- Let's write a class that manages lecture information
- Properties, constructor, and GetCode() are given in lecture.hpp file (**do not fix this file**)

```
enum Grade { A, B, C, NA };
...
class Lecture {
private:
    char code[MAX_CODE_LEN + 1];
    int student_cnt;
    int ids[MAX_STUDENT];
    Grade grades[MAX_STUDENT];
public:
    Lecture(const char * _code): student_cnt(0) {
        strncpy(code, _code, MAX_CODE_LEN);
        code[MAX_CODE_LEN] = '\0';
    }
}
```

Homework 6-1: Lecture Class

- Implement the following functions in `lecture.cpp`

```
// Add a student whose ID is 'id'. The student's grade must be
// initialized as 'NA'. Store the information of the student at
// 'ids[student_cnt]' and 'grades[student_cnt]'. After adding the
// student, increase 'student_cnt' by one and return true. If the
// lecture is already full or 'id' is already present in this
// lecture, don't add the student and just return false.
bool AddStudent(int id);

// Update the grade of the student whose ID is 'id' into 'grade'. If
// such a student doesn't exist, just return false.
bool SetGradeOf(int id, Grade grade);

// Get the grade of the student whose ID is 'id', and store it at
// '*pGrade'. If such a student doesn't exist, just return false.
bool GetGradeOf(int id, Grade *pGrade);
```

Homework 6-1: Lecture Class

- In `main.cpp`, you can find a test function for Lecture
 - You can freely change this function to test your class implementation in various ways

```
void test() {
    Lecture lec1("CSE2015");
    Lecture lec2("CSE2035");
    Grade grade;

    // Test with the first lecture.
    lec1.AddStudent(20220001);
    lec1.AddStudent(20220002);
    lec1.SetGradeOf(20220002, A);

    // This should work and print "A" as grade.
    if (lec1.GetGradeOf(20220002, &grade)) {
        cout << "Grade of 20220002: " << GradeToString(grade) << endl;
    }
    ...
}
```

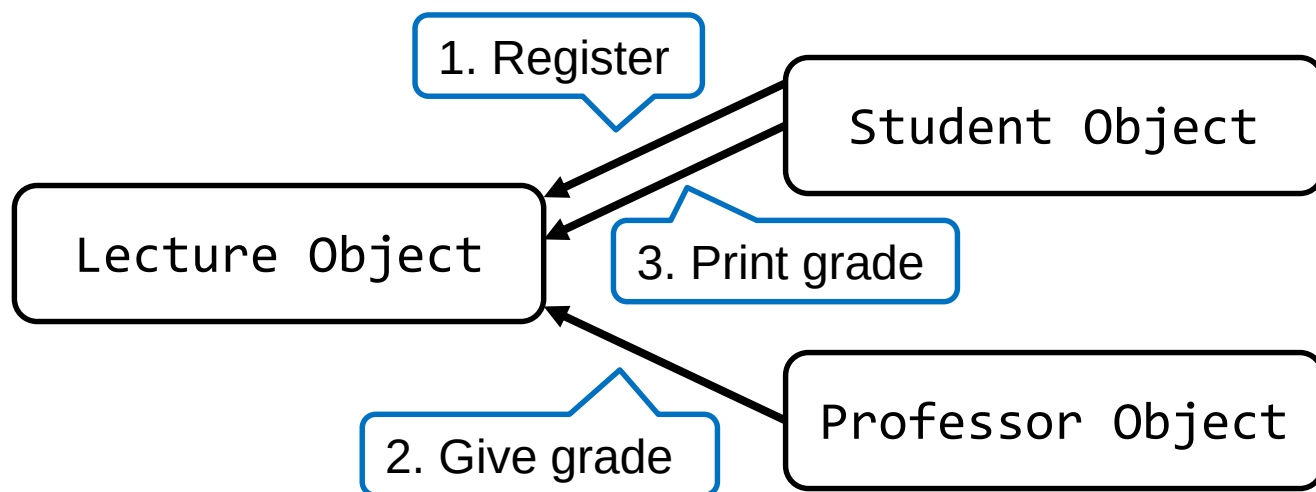
Homework 6-2: Lecture Management

- Now let's write classes that interact with Lecture
- First, copy your hw6-1/lecture.cpp file into hw6-2/lecture.cpp
- UnivMem class is defined in univmem.hpp (do not fix this file) and this class will be inherited by other classes

```
class UnivMem {
private:
    int id;
    char name[MAX_NAME_LEN + 1];
public:
    UnivMem(int _id, const char* _name) { ... }
    int GetID() { ... }
    const char* GetName() { ... }
    void Print() { ... }
```

Homework 6-2: Lecture Management

- We will define two classes (Student and Professor) that inherit UnivMem class
- A Student object can (1) register to the lectures and (2) print out the grades from each lecture
- A Professor object can grade students in a lecture



Homework 6-2: Lecture Management

- First, I'm giving you Professor class definition
- Professor class is defined in professor.hpp file (**do not fix this file**) by inheriting UnivMem
 - Properties, constructor, and GradeStudent() are defined

```
class Professor: public UnivMem {
private:
    Lecture *lecture;
public:
    Professor(int _id, const char* _name, Lecture * _lecture)
    { ... }

    // Update the student's grade in 'lecture'. Set the grade of
    // the student with 'student_id' into 'student_grade'.
    // Return false if such a student does not exist in 'lecture'.
    bool GradeStudent(int student_id, Grade student_grade) { ... }
```

Homework 6-2: Lecture Management

- Student class is partially defined in `student.hpp` (**do not fix this file**) by inheriting `UnivMem`
 - Properties and constructor are given
 - 'lecture_cnt' stores how many lectures this student is taking
 - 'lectures' stores pointers to the lecture objects that this student is taking
 - A student can take up to `MAX_LECTURE` lectures

```
class Student: public UnivMem {  
private:  
    int lecture_cnt;  
    Lecture* lectures[MAX_LECTURE];  
public:  
    Student(int _id, const char* _name) { ... }  
    ...  
}
```


Homework 6-2: Lecture Management

- Implement the following functions in `student.cpp`

```
// Print the grades of this student. For each of the lecture,
// print its lecture code and the student's grade in the
// following format:
// "<lecture code> : <grade string>\n"
// You can use GradeToString() function provided in lecture.hpp
void PrintGrade();

// If this student can take more lectures, try to add this
// student to 'lec'. Return false if this registration was
// unsuccessful. If the registration was successful, update
// 'lectures' and 'lecture_cnt' properties to reflect the
// registration, and then return true.
bool Register(Lecture *lec);
```

Homework 6-2: Lecture Management

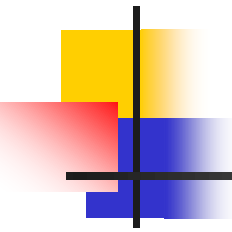
- In `main.cpp`, you can find a test function
 - You can freely change this function to test your class implementation in various ways

```
void test() {
    Lecture lec1("CSE2015");
    Lecture lec2("CSE2035");
    Professor p(102233, "Jaeseung", &lec1);
    Student s1(20220001, "Alice");
    Student s2(20220002, "Brian");

    s1.Register(&lec1);
    s1.Register(&lec2);
    p.GradeStudent(20220001, B);
    s1.Print();
    s1.PrintGrade();
    ...
}
```

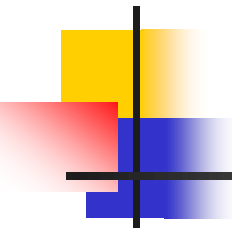
(Expected Output)

```
Alice(20220001)
CSE2015 : B
CSE2035 : NA
```

A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Submission Guideline

- **Submit the completed `lecture.cpp` and `student.cpp` files to the “Week 6 Lab Assignment” page in cyber campus**
 - You can fix `main.cpp` to test your classes, but don't submit it
 - Do not zip the directory, just upload these files directly
 - If the submitted file doesn't compile with the "make" command, cannot give you any point
- **Deadline is 10/19 (Wednesday) 23:59**
 - I'm giving you two weeks
 - There will be no new assignment next week
- **Late submission is allowed until 10/21 (Friday) 23:59**
 - 20% deduction

A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Q & A Session
